L14 - LoRA 高效微调

> *"四两拨千斤的微调艺术"*

📌 本节目标

1. 理解全参微调的问题和 LoRA 的动机
2. **掌握 LoRA 的数学原理**（面试必考）
3. 理解 LoRA 的参数量计算和效率对比
4. 能读懂 MiniMind 纯手写的 LoRA 实现
5. 了解 LoRA 的应用场景和 QLoRA

📚 前置知识

- L13 监督微调 SFT（理解 SFT 的目标和流程）
- 线性代数基础（矩阵乘法、矩阵的秩）
- 了解神经网络中线性层的参数

---

1. 全参微调的问题

1.1 什么是全参微调?

全参微调（Full Fine-Tuning）就是在 SFT 时更新模型的**所有参数**。对于 MiniMind 的 64M 参数来说这还可以接受，但对于大模型来说问题就来了：

模型	参数量	FP16 权重大小	训练显存（估算）
MiniMind	64M	~128MB	~1GB
LLaMA-7B	7B	~14GB	~56GB
LLaMA-70B	70B	~140GB	~560GB

1.2 全参微调的三大问题

1. **计算量巨大**：需要为所有参数计算梯度并更新
2. **显存不够**：训练显存 ≈ 4 × 模型权重大小（参数 + 梯度 + 优化器状态）
3. **容易过拟合**：参数量远大于 SFT 数据量时，模型容易记忆训练集
4. **存储成本高**：每个下游任务都需要保存一份完整的模型权重

1.3 能否只训练一小部分参数?

最直觉的想法：冻结大部分参数，只训练最后几层。但这种方式效果有限，因为每一层都可能需要适配新任务。

LoRA 提供了一个更优雅的解决方案。

---

2. LoRA 的核心思想

2.1 核心假设

LoRA（Low-Rank Adaptation）的核心假设是：

> **模型在适配下游任务时，权重的变化量 ΔW 是低秩的。**

换句话说，虽然权重矩阵 W 本身是高维的（比如 768×768），但从预训练到微调，权重的**变化** ΔW 可以用一个低秩矩阵很好地近似。

2.2 低秩分解

如果一个 $d \times d$ 的矩阵 ΔW 的秩为 r（且 $r \ll d$），那么它可以分解为两个小矩阵的乘积：

$\Delta W = B \times A$

其中： - $B \in \mathbb{R}^{d \times r}$ - $A \in \mathbb{R}^{r \times d}$

参数量从 d^2 降低到 $2dr$。当 $r \ll d$ 时，这是一个巨大的压缩。

2.3 LoRA 的工作方式

LoRA 的做法：

1. **冻结**原始预训练权重 W（不更新）

2. 在旁路添加两个小矩阵 B 和 A
3. 前向传播时：$W^{'} = W + BA$

```
      ┌──────────────┐
x ──→ │  W (冻结)    │ ──→  Wx
|     └──────────────┘        |
|                             + ──→ 输出 = Wx + BAx
|     ┌───┐   ┌───┐           |
└───→ | A | → | B | ──→  BAx
      └───┘   └───┘

      r×d     d×r
      (可训练)
```

训练时只更新 A 和 B，原始权重 W 完全不变。

2.4 初始化策略

- **A 矩阵**：使用随机初始化（通常是高斯分布）
- **B 矩阵**：初始化为全零

为什么 B 初始化为零？因为这样训练开始时 $BA = 0$，模型的输出和原始模型完全一致，LoRA 不会干扰预训练学到的知识。随着训练进行，B 逐渐学到有意义的值。

2.5 缩放因子

实际使用时还有一个缩放因子 α：

$$W^{'} = W + \frac{\alpha}{r} \cdot BA$$

α / r

控制 LoRA 的"影响力"。通常 设为 的 1-2 倍，使缩放因子接近 1。

---

3. 参数量对比

3.1 全参 vs LoRA

以 MiniMind 中一个典型的线性层为例：

原始线性层（768 → 768）：

参数量 $= 768 \times 768 = 589,824$

LoRA（r=8）：

参数量 $= 768 \times 8 + 8 \times 768 = 6,144 + 6,144 = 12,288$

压缩比：

$$\frac{12,288}{589,824} \approx 2.1\%$$

只训练 **2%** 的参数！

3.2 不同秩 r 的参数量

秩 r	LoRA 参数量	占原始比例
1	1,536	0.26%
4	6,144	1.04%
8	12,288	2.08%
16	24,576	4.17%
32	49,152	8.33%
64	98,304	16.67%

3.3 秩 r 如何选择？

- **r 太小**：表达能力不足，无法充分适配下游任务
- **r 太大**：参数量增加，失去 LoRA 的效率优势
- **经验值**：r = 4~16 通常就够了
- **复杂任务**用更大的 r，简单任务用更小的 r

原始论文中的实验表明，对于大多数 NLP 任务，r = 4 就能达到接近全参微调的效果。

---

4. LoRA 加在哪些层？

4.1 通常的选择

LoRA 不需要加在模型的每一层上。最常见的做法是加在 **Attention 层的投影矩阵**上：

投影层	维度	说明
Q 投影（Wq）	768 → 768	Query 投影

投影层	维度	说明
K 投影（Wk）	768 → 384（GQA）	Key 投影
V 投影（Wv）	768 → 384（GQA）	Value 投影
O 投影（Wo）	768 → 768	Output 投影

有些实现也会在 FFN 层加 LoRA，但性价比不如 Attention 层。

4.2 为什么 Attention 层更有效?

Attention 层是模型学习"关注什么"的核心。不同下游任务需要模型关注不同的信息，因此 Attention 层的权重变化最大。在这些层加 LoRA，能以最少的参数实现最大的适配效果。

---

5. MiniMind 的 LoRA 实现

5.1 亮点：纯手写

MiniMind 的 LoRA 实现**不依赖 peft 库**，完全从零手写。这是学习 LoRA 原理的绝佳素材。

5.2 关键文件

- `model/model_lora.py`：LoRA 模型定义
- `trainer/train_lora.py`：LoRA 训练脚本
- `scripts/convert_model.py`：LoRA 权重合并

5.3 LoRA 线性层的实现

```
class LoRALinear(nn.Module):
    """带 LoRA 旁路的线性层"""
    def __init__(self, in_features, out_features, r=8, alpha=16):
        super().__init__()
        self.linear = nn.Linear(in_features, out_features, bias=False)
        self.linear.weight.requires_grad = False  # 冻结原始权重

        # LoRA 旁路
        self.lora_A = nn.Parameter(torch.randn(r, in_features) * 0.01)
        self.lora_B = nn.Parameter(torch.zeros(out_features, r))
        self.scaling = alpha / r

    def forward(self, x):
        # 原始路径 + LoRA 路径
        base_output = self.linear(x)
```

```
        lora_output = (x @ self.lora_A.T @ self.lora_B.T) * self.scaling
        return base_output + lora_output
```

核心就这么简单！冻结原始权重，加一个 A 和 B 的旁路，训练时只更新 A 和 B。

5.4 LoRA 的训练流程

`train_lora.py` 与 `train_full_sft.py` 的主要区别：

```
# 1. 加载预训练模型
base_model = MiniMindForCausalLM(config)
base_model.load_state_dict(torch.load('pretrain_checkpoint.pt'))

# 2. 将模型转换为 LoRA 模型（替换线性层）
lora_model = convert_to_lora(base_model, r=8, alpha=16)

# 3. 冻结所有原始参数，只训练 LoRA 参数
for name, param in lora_model.named_parameters():
    if 'lora' not in name:
        param.requires_grad = False

# 4. 查看可训练参数量
trainable = sum(p.numel() for p in lora_model.parameters() if p.requires_grad)
total = sum(p.numel() for p in lora_model.parameters())
print(f"Trainable: {trainable:,} / {total:,} = {100*trainable/total:.2f}%")

# 5. 训练循环（与 SFT 相同）
optimizer = AdamW(filter(lambda p: p.requires_grad, lora_model.parameters()), lr=1e-4)
```

5.5 LoRA 权重合并

训练完成后，可以将 LoRA 权重合并回原始权重，推理时无额外开销：

$$W_{\text{merged}} = W + \frac{\alpha}{r} \cdot BA$$

```
# scripts/convert_model.py 的核心逻辑
def merge_lora_weights(model):
    for module in model.modules():
        if isinstance(module, LoRALinear):
            # 将 LoRA 权重合并到原始权重
            delta_w = (module.lora_B @ module.lora_A) * module.scaling
            module.linear.weight.data += delta_w
    return model
```

合并后的模型和普通模型完全相同，推理时不需要任何额外计算。

---

6. LoRA 的应用场景

6.1 垂直领域适配

用 LoRA 将通用模型适配到特定领域：

```
通用模型 + 医疗LoRA → 医疗问答模型
通用模型 + 法律LoRA → 法律咨询模型
通用模型 + 编程LoRA → 代码生成模型
```

每个 LoRA 适配器只有几 MB，而基础模型有几十 GB。

6.2 自我认知定制

```
通用模型 + 认知LoRA → "我是小明助手"
通用模型 + 认知LoRA → "我是医疗AI"
```

6.3 多任务共存

一个基础模型可以同时配备多个 LoRA 适配器：

```
基础模型权重（共享） ← 只存一份
  ├── 任务A的LoRA权重 ← 几 MB
  ├── 任务B的LoRA权重 ← 几 MB
  └── 任务C的LoRA权重 ← 几 MB
```

推理时根据请求切换不同的 LoRA，极大节省存储和部署成本。

---

7. QLoRA 简介

7.1 量化 + LoRA

QLoRA 在 LoRA 的基础上进一步优化：将冻结的原始权重**量化到 4-bit**（NF4 格式），进一步降低显存占用。

```
原始:   FP16权重（冻结） + FP16 LoRA权重（可训练）
QLoRA:  4-bit权重（冻结） + FP16 LoRA权重（可训练）
```

7.2 QLoRA 的关键技术

1. **NF4 量化**：一种专为正态分布设计的 4-bit 量化方式
2. **双重量化**：连量化常数也量化，进一步节省显存
3. **分页优化器**：利用 CPU 内存处理优化器状态

7.3 QLoRA 的效果

QLoRA 让你可以在消费级 GPU（24GB）上微调 7B 甚至 13B 的模型，且效果接近全参微调。MiniMind 模型本身就很小，不太需要 QLoRA，但对于更大的模型这是非常实用的技术。

---

8. LoRA vs 全参微调的对比

维度	全参微调	LoRA
可训练参数	100%	1-5%
显存占用	高	低（冻结权重不需要梯度）
训练速度	慢	快
过拟合风险	高	低（天然正则化）
效果上限	最高	接近全参（通常 95%+）
部署灵活性	低（每任务一份模型）	高（共享基座，切换LoRA）
适用场景	数据充足、追求极致效果	数据较少、需要快速适配

🎤 面试考点

Q1: 请解释 LoRA 的原理和数学公式。（必考）

答：LoRA 的核心假设是模型在适配下游任务时，权重变化量 ΔW 是低秩的。因此可以将 ΔW 分解为两个低秩矩阵的乘积：$\Delta W = BA$，其中 $B \in \mathbb{R}^{d \times r}$，$A \in \mathbb{R}^{r \times d}$，$r \ll d$。训练时冻结原始权重 W，只训练 A 和 B。前向传播时：$h = Wx + \frac{\alpha}{r}BAx$。初始化时 B=0 保证训练开始时不改变原始模型行为。

Q2: LoRA 的秩 r 如何选择?

答：r 的选择取决于任务复杂度：- 简单任务（如情感分类）：r = 1~4 就够了 - 中等任务（如指令微调）：r = 8~16 - 复杂任务（如多领域适配）：r = 16~64 原始论文实验表明，r = 4 在大多数 NLP 任务上已经能达到接近全参微调的效果。过大的 r 会增加参数量，失去 LoRA 的效率优势。

Q3: LoRA vs 全参微调的优劣?

答：- **LoRA 优势**：(1) 只训练 2% 参数，显存和计算量大幅降低；(2) 天然正则化，不易过拟合；(3) 多任务可共享基座模型；(4) 权重可合并，推理无额外开销 - **LoRA 劣势**：(1) 表达能力受限于秩 r，效果上限低于全参；(2) 某些需要大幅调整模型行为的任务效果可能不足 - **选择建议**：数据少或资源有限用 LoRA，数据充足且追求极致效果用全参

Q4: 为什么 LoRA 的 B 矩阵初始化为零?

答：确保训练开始时 $BA = 0$，LoRA 旁路不会改变原始模型的输出。这样做有两个好处：(1) 保留预训练学到的知识，从一个好的起点开始微调；(2) 训练更稳定，不会因为随机初始化的 LoRA 破坏原始模型的表现。

Q5: LoRA 通常加在模型的哪些层？为什么?

答：通常加在 Attention 层的 Q、K、V、O 投影矩阵上。因为 Attention 层负责学习"关注什么信息"，不同下游任务的关注模式差异最大，因此在这些层加 LoRA 的性价比最高。部分实现也会在 FFN 层加 LoRA，但增加的参数量较大，边际效果递减。

Q6: LoRA 权重合并后推理时有额外开销吗?

答：没有。合并后 $W_{\text{merged}} = W + \frac{\alpha}{r}BA$，模型结构和普通模型完全相同，推理时没有任何额外计算。这是 LoRA 相比其他 adapter 方法的一大优势。

✅ 自测题

1. 计算：对于一个 1024×1024 的线性层，使用 r=16 的 LoRA，可训练参数量是多少？占原始参数的百分比?
2. 手动推导：给定 $x \in \mathbb{R}^{1\times 768}$，$W \in \mathbb{R}^{768\times 768}$，$A \in \mathbb{R}^{8\times 768}$，$B \in \mathbb{R}^{768\times 8}$，写出 LoRA 前向传播的完整计算过程和每步的维度。
3. 为什么说 LoRA 具有"天然正则化"效果?
4. 如果 LoRA 的 A 和 B 都初始化为零会怎样?

5. 实现一个简单的 LoRA 线性层（不看参考代码）。

⏭️ 下一节预告

L15 - 知识蒸馏：除了 LoRA，还有一种强大的技术——知识蒸馏，让小模型从大模型那里"偷师学艺"。MiniMind 同时使用了黑盒蒸馏和白盒蒸馏，我们将深入理解温度、KL 散度和蒸馏损失函数。

αr